

MTR-RSNN: Multi-Timescale Recurrent SNNs with Kinetic NMDA Gating and Online Learning

Riyane El Qoqui
Independent Researcher

June 2026

Abstract

Recurrent spiking neural networks (RSNNs) have advanced toward biological plausibility through adaptive thresholds and surrogate gradient learning, yet two gaps persist: the NMDA receptor’s Mg^{2+} block is universally modeled as a static voltage function [4], discarding its temporal dynamics, and inference is assumed to require GPU or neuromorphic hardware.

This work introduces MTR-RSNN, a multi-timescale recurrent SNN that closes both gaps. The neuron model integrates kinetic Mg^{2+} gating via the two-state implicit Euler discretization of Kampa et al. [5], spike-frequency adaptation (ALIF), two-component after-hyperpolarization (AHP), and four semantically routed dendritic branches. Two dendritic populations with distinct somatic leak factors ($\rho_s = 0.50$ and $\rho_s = 0.90$) are coupled through a 10% sparse recurrent layer and feed a readout LIF population without AHP or NMDA gating.

Online learning combines forward e-prop eligibility traces [1] with RMSProp-normalized updates [11], a per-action advantage baseline, and astrocyte-inspired adaptive learning rates. The full pipeline is implemented in bare-metal C++/AVX2 with structure-of-arrays memory layout and Estrin-scheme Taylor kernels for the kinetic rate functions.

On an Intel i7-12650H (4.1 GHz, single P-core), `tick()` over 256 neurons completes in 1,082 cycles (4.23 cycles/neuron, IPC = 3.5, 0.00% branch misprediction); a full forward pass runs in 4,845 ns at 208,000 ticks/second. Evaluated on a real-world ultra-low-latency sequential decision-making task comprising 135 million timesteps of live market microstructure data, all three behavioral profiles converge stably under online e-prop. To the best of the author’s knowledge, MTR-RSNN is the first recurrent SNN to combine kinetic NMDA gating with online eligibility-trace learning, achieving sub-5 μs inference on commodity hardware versus 0.33 ms on Loihi 2 ASIC [2], without neuromorphic silicon.

Contents

1	Introduction	4
2	Related Work	5
3	Mathematical Formulations	6
3.1	LIF Discretized	6
3.2	Spike-Frequency Adaptation (ALIF)	6
3.3	AMPA/NMDA Dual Kinetics	6
3.4	Kinetic Mg^{2+} Gating (Kampa et al., 2004)	6
3.5	Branch Output	7
3.6	After-Hyperpolarization (AHP)	7
3.7	Somatic Integration	7
3.8	Graded Spike	7
3.9	Triangular Surrogate Gradient	7
3.10	Eligibility Trace (E-prop)	7
3.11	RMSProp-Normalized Weight Update	8
3.12	Per-Action Advantage	8
3.13	Astrocyte Neuromodulation	8
4	Biophysical Neuron Model	8
4.1	From LIF to ALIF	8
4.2	AMPA/NMDA Dual Kinetics	9
4.3	Kinetic Mg^{2+} Gating	9
4.4	After-Hyperpolarization (AHP)	11
4.5	Graded Spikes and Readout Population	12
5	Network Architecture	12
5.1	Multi-Timescale Three-Population Architecture	12
5.2	Sparse Recurrent Connectivity	13
5.3	Semantic Dendritic Routing	13
6	Learning Algorithm	14
6.1	Forward E-prop with LIF Eligibility Trace	14
6.2	Triangular Surrogate Gradient	14
6.3	RMSProp-Normalized Weight Update	15
6.4	Per-Action Advantage Baseline	15
6.5	Astrocyte Neuromodulation	16
7	Implementation	16
7.1	Structure-of-Arrays Memory Layout	16
7.2	Dual Estrin ILP for Kinetic Rate Functions	17
7.3	<code>tick()</code> Inner Loop	17
7.4	<code>forward_dense()</code> Column-Major GEMV	18
7.5	<code>forward_sparse()</code> Branchless Sparse GEMV	19
7.6	<code>tick_d0()</code> Single-Branch Kernel	19
7.7	<code>predict()</code> Full Pipeline	20
8	Benchmarks	21
9	Limitations	23

1 Introduction

Recurrent spiking neural networks (RSNNs) have emerged as a biologically plausible substrate for sequential computation, combining spike-based communication with adaptive dynamics such as spike-frequency adaptation and dendritic integration [1, 8, 12]. Despite this progress, the synaptic gating model underlying NMDA receptor currents has remained static across the SNN literature: every major implementation inherits the equilibrium voltage function of Jahr and Stevens [4], which assumes instantaneous Mg^{2+} unblock and is valid only under sustained depolarization. Vargas-Caballero and Robinson [10] demonstrated that Mg^{2+} unblock is bi-exponential with time constants of 0.68 ms and 22.7 ms, rendering the static approximation incorrect during the rapid voltage transients that define spiking activity. Kampa et al. [5] subsequently parameterized the full two-state kinetic model, with an electrical distance $\delta = 0.80$ and sub-millisecond reblocking, providing the mechanistic foundation that static models discard.

The consequences extend beyond biological fidelity. To the best of the author’s knowledge, no published RSNN combines kinetic Mg^{2+} gating with online eligibility-trace learning: existing multi-compartment SNNs with dendritic dynamics rely on backpropagation through time and GPU execution [3, 9, 12, 13], which precludes deployment in ultra-low-latency settings where inference must complete within microseconds on commodity hardware. Neuromorphic ASICs such as Loihi 2 achieve sub-millisecond inference [2], but require specialized silicon unavailable in standard compute environments. Commodity x86-64 CPUs, by contrast, offer deterministic latency, cache hierarchies amenable to structure-of-arrays layouts, and AVX2 vector units capable of processing eight neurons per instruction, making them a natural substrate for ultra-low-latency sequential decision-making when the network and its kernels are co-designed accordingly.

This work makes the following contributions:

- **Kinetic Mg^{2+} gating in a recurrent SNN.** The two-state implicit Euler discretization of Kampa et al. [5] is integrated into a fully recurrent network with online learning, replacing the static $B(V)$ function [4] with a per-neuron persistent state $B[t] \in [0, 1]$ that evolves each tick.
- **Multi-timescale three-population architecture.** A fast hidden population ($\rho_s = 0.50$, 256 neurons, 4 dendritic branches, sparse recurrent), a slow integrator population ($\rho_s = 0.90$, 64 neurons, single branch, no recurrence), and a readout LIF population (8 neurons) provide three distinct temporal integration windows within a single forward pass.
- **Online e-prop with RMSProp trace normalization and per-action advantage.** Forward eligibility traces [1] are normalized by a per-synapse RMS second moment [11], initialized at unity to avoid early divergence. A per-action EMA baseline provides neuron-level advantage estimates without a learned critic network.
- **Sparse GEMV with precompiled block index.** The 10% sparse recurrent layer is executed via a precompiled list of non-zero 8-element blocks, enabling AVX2 broadcast-fmadd without gather instructions and branch-prediction-free tick execution (0.00% misprediction rate).
- **tick_d0: single-branch kernel for slow integrators.** A specialized tick variant for single-branch populations without recurrent input eliminates the three additional dendritic branch pairs and the recurrent GEMV, reducing the per-neuron instruction count from 98 to 69 inner-loop instructions.

On an Intel i7-12650H at 4.1 GHz (single P-core, performance governor), `tick()` over 256 neurons with kinetic NMDA gating and AHP completes in 1,082 cycles (4.23 cycles/neuron, IPC = 3.5, 0.00% branch misprediction); a complete forward pass runs in 4,845 ns at 208,000 ticks/second (IPC = 4.2). Evaluated on a real-world ultra-low-latency sequential decision-making task comprising 135 million timesteps of live market microstructure data, all three behavioral profiles converge stably under online e-prop without gradient clipping or batch normalization.

2 Related Work

Surrogate gradient learning in SNNs. Zenke and Ganguli [11] introduced SuperSpike, establishing the surrogate gradient framework for multilayer SNNs and the RMaxProp per-synapse normalization that directly motivates the $\mathcal{E}_{ji}^2$ trace used here. Bellec et al. [1] extended this to recurrent networks via e-prop, proving that the exact RSNN gradient decomposes into a sum of local eligibility traces weighted by a top-down learning signal, eliminating the need for backpropagation through time. For LIF neurons the trace reduces to $\mathcal{E}_{ji}[t+1] = \psi_j^{t+1} \cdot z_i^t$ [1], which is the form implemented here; the full ALIF trace carrying the adaptation term $\beta \cdot \varepsilon_{ji,a}$ is not used, a limitation discussed in Section 9. Noè et al. [7] recently combined e-prop with a predictive coding objective in a point-neuron RSNN, converging in 23 epochs versus 80 for truncated BPTT on a sine-wave generation task; their work uses the full ALIF trace but no dendritic compartments, NMDA gating, or bare-metal CPU implementation.

Dendritic computation in SNNs. Poirazi et al. [8] demonstrated that a pyramidal neuron with several dozen thin terminal dendrites behaves as a two-layer neural network, establishing the computational role of dendritic nonlinearities that motivates multi-compartment SNN models. Zheng et al. [12] proposed DH-SNN, in which each dendritic branch carries a heterogeneous time constant α_d learned by BPTT, achieving 92.1% on SHD with 0.05M parameters. DH-SNN is the structurally closest prior work: it uses multi-compartment recurrent neurons and multi-timescale dynamics, but relies on BPTT, GPU execution, and linear RC dendritic filters without NMDA gating or AHP. Huang et al. [3] introduced DendSNN, replacing the RC filter with a wavelet neural network aggregation across branch compartments, compiled via Triton for GPU throughput within 8% of a plain LIF kernel; the model is feedforward, trained with BPTT, and carries no ionic channel model. Zhou et al. [13] extended DH-SNN with a combined exponential-linear membrane decay gated by a trainable factor and an adaptive firing threshold, improving SHD accuracy to 90.12% while remaining feedforward and BPTT-trained. None of these works model NMDA receptor kinetics or support online learning.

Recurrent SNNs for sequential decision-making. Sun et al. [9] proposed Spiking World Model, a multi-compartment recurrent SNN with apical and basal dendrites integrated into a Dreamer-style model-based RL pipeline, reaching 90.4% of GRU performance on 19 DeepMind Control Suite tasks. Their dendritic gating uses a sigmoid function of the apical voltage, which is functionally analogous to a static $B(V)$ modulation: it carries no voltage-dependent kinetic state, no $[\text{Mg}^{2+}]$ dependence, and no implicit Euler integration. Training relies on BPTT and GPU execution throughout.

Neuromorphic hardware and CPU inference. Hajizada et al. [2] deployed a continual-learning SNN on Intel Loihi 2, achieving 0.33 ms inference latency on a dedicated neuromorphic ASIC with 128 neurocores via a local three-factor Oja learning rule; this is not e-prop and the comparison point is hardware latency, not learning algorithm. The Open Neuromorphic benchmark [6] reports SpikingJelly CuPy on an RTX 4090 completing a forward-backward pass over 16k LIF neurons in 0.26 s (500 timesteps, batch 16), establishing the GPU baseline for SNN throughput. Neither setting targets bare-metal CPU inference: Loihi 2 requires specialized silicon and the SpikingJelly benchmark has no CPU-optimized equivalent for multi-compartment neurons. MTR-RSNN addresses this gap with AVX2 structure-of-arrays kernels that achieve 4.23 cycles/neuron on a commodity i7-12650H.

Table 1: Comparison of MTR-RSNN with related SNN architectures.

Method	Kin. NMDA	Multi-comp.	Rec.	E-prop	CPU
SuperSpike [11]	×	×	×	×	×
E-prop [1]	×	×	✓	✓	×
Pred. E-prop [7]	×	×	✓	✓	×
DH-SNN [12]	×	✓	✓	×	×
DendSNN [3]	×	✓	×	×	×
CD-DT-DSNN [13]	×	✓	×	×	×
Spiking-WM [9]	×	✓	✓	×	×
CLP-SNN/Loihi 2 [2]	×	×	×	×	×
MTR-RSNN	✓	✓	✓	✓	✓

Kin. NMDA : kinetic Mg^{2+} gating. *Multi-comp.* : multi-compartment. *Rec.* : recurrent. *E-prop* : online without BPTT. *CPU* : bare-metal kernel.

3 Mathematical Formulations

3.1 LIF Discretized

The membrane potential of a leaky integrate-and-fire neuron is governed by the RC circuit $\tau_m dV/dt = -V + I$, discretized with timestep Δt :

$$V[t] = \rho V[t-1] + I[t], \quad \rho = e^{-\Delta t/\tau_m} \quad (1)$$

3.2 Spike-Frequency Adaptation (ALIF)

$$A[t] = \alpha_a A[t-1] + \beta_a z[t-1] \quad (2)$$

$$\theta[t] = \theta_0 + A[t] \quad (3)$$

where $\alpha_a = 0.95$, $\beta_a = 0.10$, $\theta_0 = 1.0$.

3.3 AMPA/NMDA Dual Kinetics

Each dendritic branch maintains two synaptic current traces with distinct time constants ($\rho_{\text{ampa}} = 0.80$, $\rho_{\text{nmda}} = 0.98$):

$$V_{\text{AMPA},k}[t] = \rho_{\text{ampa}} V_{\text{AMPA},k}[t-1] + I_k[t] \quad (4)$$

$$V_{\text{NMDA},k}[t] = \rho_{\text{nmda}} V_{\text{NMDA},k}[t-1] + I_k[t] \quad (5)$$

3.4 Kinetic Mg^{2+} Gating (Kampa et al., 2004)

The unblocking probability $B[t] \in [0, 1]$ evolves via voltage-dependent transition rates:

$$\alpha[t] = k_u \exp(\delta V_s[t]) \quad (6)$$

$$\beta[t] = k_r [Mg^{2+}] \exp(-(1-\delta) V_s[t]) \quad (7)$$

$$B[t+1] = \frac{B[t] + \alpha[t]}{1 + \alpha[t] + \beta[t]} \quad (8)$$

with $\delta = 0.80$ [5], $[\text{Mg}^{2+}] = 1 \text{ mM}$, $k_u = 0.05$, $k_r = 0.08$, $B[0] = 0.4$. Equation (8) is the implicit Euler discretization of the two-state kinetic ODE, guaranteeing $B[t] \in [0, 1]$ for all t . In implementation, $\exp(\cdot)$ is approximated by a 4th-order Taylor expansion valid to within 0.02% for $|\delta V_s| < 0.4$, which covers the normalized voltage range of the hidden population.

3.5 Branch Output

$$d_k[t] = V_{\text{AMPA},k}[t] + g_{\text{NMDA}} \cdot B[t] \cdot V_{\text{NMDA},k}[t] \quad (9)$$

with $g_{\text{NMDA}} = 0.50$.

3.6 After-Hyperpolarization (AHP)

Two-component AHP with fast ($\rho_{\text{ahp},f} = 0.90$) and slow ($\rho_{\text{ahp},s} = 0.995$) K^+ conductances:

$$g_{\text{fast}}[t] = \rho_{\text{ahp},f} g_{\text{fast}}[t-1] + \Delta g_f \mathbf{1}_{V_s[t-1] \geq \theta[t-1]} \quad (10)$$

$$g_{\text{slow}}[t] = \rho_{\text{ahp},s} g_{\text{slow}}[t-1] + \Delta g_s \mathbf{1}_{V_s[t-1] \geq \theta[t-1]} \quad (11)$$

$$I_{\text{AHP}}[t] = (g_{\text{fast}}[t] + g_{\text{slow}}[t]) (V_s[t-1] - E_K) \quad (12)$$

with $\Delta g_f = 0.30$, $\Delta g_s = 0.10$, $E_K = -0.15$.

3.7 Somatic Integration

The hidden population ($\rho_s = 0.50$) integrates four dendritic branches and recurrent input:

$$V_s[t] = \rho_s V_s[t-1] + \sum_{k=0}^3 d_k[t] + I_{\text{rec}}[t] - I_{\text{AHP}}[t] \quad (13)$$

The slow population ($\rho_s = 0.90$) receives a single dendritic branch and no recurrent input:

$$V_s[t] = \rho_s V_s[t-1] + d_0[t] - I_{\text{AHP}}[t] \quad (14)$$

3.8 Graded Spike

$$z[t] = \begin{cases} \max(V_s[t] - \theta[t], 1) & \text{if } V_s[t] \geq \theta[t] \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

On spike, $V_s[t]$ is reset to $E_K = -0.15$.

3.9 Triangular Surrogate Gradient

$$h'[t] = \max(0, 1 - \beta_{\text{sg}} |V_{\text{pre}}[t] - \theta[t]|) \quad (16)$$

where $V_{\text{pre}}[t]$ is the pre-reset soma voltage.

3.10 Eligibility Trace (E-prop)

$$\mathcal{E}_{ji}[t] = \rho_e \mathcal{E}_{ji}[t-1] + z_j[t-1] \cdot h'_i[t] \quad (17)$$

with $\rho_e = \rho_s$ of the postsynaptic population.

3.11 RMSProp-Normalized Weight Update

The RMS-normalized form of the global learning signal is defined as:

$$\overline{L^2}[t] = 0.999 \overline{L^2}[t-1] + 0.001 L[t]^2 \quad (18)$$

$$\bar{L}[t] = \text{clamp}\left(\frac{L[t]}{\sqrt{\overline{L^2}[t] + \varepsilon}}, -3, 3\right) \quad (19)$$

$\bar{L}[t]$ is tracked for monitoring purposes; the weight update (21) uses the raw signal $L[t]$ directly. The eligibility trace second moment and weight update are then:

$$\mathcal{E}_{ji}^2[t] = 0.999 \mathcal{E}_{ji}^2[t-1] + 0.001 \mathcal{E}_{ji}[t]^2, \quad \mathcal{E}_{ji}^2[0] = 1 \quad (20)$$

$$\Delta W_{ji}[t] = \eta_{\text{dyn}}[t] L[t] \frac{\mathcal{E}_{ji}[t]}{\sqrt{\mathcal{E}_{ji}^2[t] + \varepsilon}} - \lambda W_{ji}[t] \quad (21)$$

3.12 Per-Action Advantage

$$\hat{V}_i[t] = (1 - \alpha_c) \hat{V}_i[t-1] + \alpha_c r[t] \quad (22)$$

$$a_i[t] = r[t] - \hat{V}_i[t] \quad (23)$$

$$r[t] = r_{\text{fill}}[t] - \lambda q[t]^2 \quad (24)$$

where $r_{\text{fill}}[t]$ is the realized profit from filled limit orders, $q[t]$ is the current inventory position, λ is the risk penalty, and $\alpha_c \in [0.01, 0.019]$ decreases with market stress.

3.13 Astrocyte Neuromodulation

$$c[t] = 0.99 c[t-1] + 0.01 \cdot \sigma_{\text{vol}}[t] \cdot \bar{v}[t] \quad (25)$$

$$\eta_{\text{dyn}}[t] = \eta (1 + \kappa \cdot \max(0, c[t] - c_{\text{thresh}})) \quad (26)$$

where $\sigma_{\text{vol}}[t]$ is the short-term volatility, $\bar{v}[t]$ is the normalized volume anomaly, $c[t]$ is the resulting EMA market stress signal, $\kappa = 5.0$, and c_{thresh} is profile-dependent (0.02–0.08).

4 Biophysical Neuron Model

At each tick t , the neuron executes the following operations in order: the SFA trace $A[t]$ is updated from the previous graded spike (2)–(3); the kinetic Mg^{2+} unblock probability $B[t]$ is stepped forward via implicit Euler (8); the synaptic traces $V_{\text{AMPA},k}[t]$ and $V_{\text{NMDA},k}[t]$ are updated and the branch outputs $d_k[t]$ computed (4)–(9); the AHP conductances are decayed and $I_{\text{AHP}}[t]$ computed (10)–(12); the somatic potential $V_s[t]$ integrates all dendritic and recurrent currents (13); and a graded spike $z[t]$ is emitted if $V_s[t] \geq \theta[t]$, followed by reset to E_K and AHP conductance bumps (15). Figure 2 illustrates the effect of step 3 on a representative spike train.

4.1 From LIF to ALIF

The standard leaky integrate-and-fire neuron discretizes the RC membrane equation $\tau_m dV/dt = -V + I$ with forward Euler at timestep Δt , yielding the leak factor $\rho = e^{-\Delta t/\tau_m} \in (0, 1)$ (1). A pure LIF neuron fires at a rate determined solely by the input current, with no mechanism to reduce firing during sustained stimulation. Biological pyramidal neurons exhibit spike-frequency adaptation (SFA): successive spikes

within a burst become progressively harder to elicit, a property mediated in part by slow K^+ currents that accumulate with activity [1].

The adaptive LIF (ALIF) model captures this by coupling a threshold adaptation variable $A[t]$ to the spike output (2):

$$A[t] = \alpha_a A[t - 1] + \beta_a z[t - 1], \quad \theta[t] = \theta_0 + A[t].$$

Each spike increments A by β_a , raising the effective threshold $\theta[t]$ (3) and suppressing subsequent firing; A then decays geometrically with rate α_a between spikes. The decay constant $\alpha_a = 0.95$ corresponds to a half-life of approximately 14 ticks, matching the slow timescale of SFA observed in cortical recordings and consistent with the value used in Bellec et al. [1]. The increment $\beta_a = 0.10$ is set to produce a threshold elevation of roughly 10% of θ_0 per spike, sufficient to reduce burst firing without silencing the neuron under sustained drive. Both values are hyperparameters in the normalized voltage domain and have no direct biological dimension.

4.2 AMPA/NMDA Dual Kinetics

Each dendritic branch models two distinct receptor populations via first-order synaptic traces (4)–(5). The AMPA trace decays with $\rho_{\text{ampa}} = 0.80$, producing a fast-decaying current that tracks recent presynaptic activity. The NMDA trace decays with $\rho_{\text{nmda}} = 0.98$, accumulating over tens of ticks and providing the slow integration window required for temporal credit assignment across the burst timescale. Both decay constants are dimensionless hyperparameters in the normalized voltage domain; they are not direct transcriptions of biological AMPA and NMDA time constants (2–5 ms and 50–150 ms respectively in physiological units) but preserve the essential ratio of fast to slow synaptic integration. Figure 1 shows the two traces on a common input consisting of a single pulse followed by a three-spike burst: at $t = 15$ (ten ticks post-pulse), V_{AMPA} has decayed to 0.107 while V_{NMDA} retains 0.817, a ratio of approximately $8\times$.

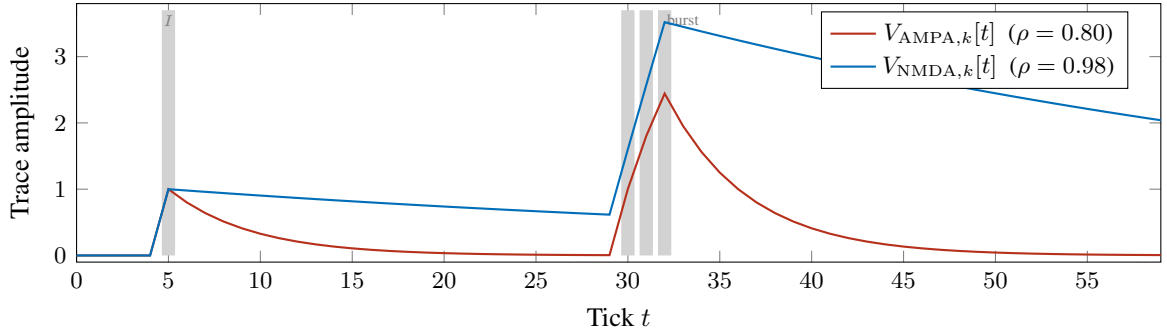


Figure 1: Fast and slow synaptic traces on a single pulse ($t = 5$) followed by a three-spike burst ($t \in \{30, 31, 32\}$). V_{AMPA} decays to baseline within 15 ticks; V_{NMDA} accumulates across the burst and persists beyond $t = 59$.

4.3 Kinetic Mg^{2+} Gating

Static vs. kinetic gating. Jahr and Stevens [4] modeled the Mg^{2+} block as the equilibrium function $B_{\text{static}}(V) = \alpha(V)/(\alpha(V) + \beta(V))$, assuming the channel is always at steady state. This is valid under sustained depolarization but fails during spiking: voltage transients are fast relative to the Mg^{2+} unbinding time constant of 0.68 ms measured by Vargas-Caballero and Robinson [10], so the channel never reaches equilibrium between spikes. The static model therefore overestimates unblock immediately after a spike and underestimates the cumulative unblock that builds across a burst. Figure 2 illustrates this directly: B_{static} jumps instantaneously to the equilibrium value at the post-reset voltage $V_s = E_K$, while $B[t]$ retains temporal state and reaches a lower trough ($B = 0.374$) after the second burst that the static model cannot represent.

Implicit Euler discretization. The two-state kinetic model of Kampa et al. [5] describes B as a continuous-time ODE:

$$\frac{dB}{dt} = \alpha(V)(1 - B) - \beta(V)B = \alpha(V) - (\alpha(V) + \beta(V))B \quad (27)$$

where $\alpha(V) = k_u \exp(\delta V)$ and $\beta(V) = k_r [\text{Mg}^{2+}] \exp(-(1 - \delta)V)$ are the voltage-dependent unblocking and reblocking rates (6)–(7). Applying the implicit Euler method with timestep $\Delta t = 1$ tick and evaluating α, β at $V[t]$ gives:

$$\frac{B[t+1] - B[t]}{1} = \alpha[t] - (\alpha[t] + \beta[t])B[t+1]$$

Solving for $B[t+1]$:

$$B[t+1](1 + \alpha[t] + \beta[t]) = B[t] + \alpha[t]$$

which yields Eq. (8) directly. Unlike explicit Euler, this scheme is unconditionally stable and guarantees $B[t+1] \in [0, 1]$ whenever $B[t] \in [0, 1]$, $\alpha[t] \geq 0$, and $\beta[t] \geq 0$, conditions always satisfied by the exponential rate functions.

Parameters and implementation. The electrical distance $\delta = 0.80$ was first characterized by Jahr and Stevens [4] from single-channel measurements and confirmed by Kampa et al. [5] in their 10-state Markov model of the binding site at 80% of the transmembrane distance. The rate constants $k_u = 0.05$ and $k_r = 0.08$ are normalized to the dimensionless voltage domain; they are not the dimensional values of Kampa et al. (expressed in ms^{-1} and $\text{mM}^{-1}\text{ms}^{-1}$ respectively) but preserve the ratio k_r/k_u and reproduce the qualitative dynamics of fast unblock and slower reblock. The channel is initialized to $B[0] = 0.4$, reflecting partial unblock at resting membrane potential in the normalized voltage domain; this is a functional approximation rather than a direct correspondence to the biological steady-state value [10]. In the inner loop, $\exp(\cdot)$ is replaced by a 4th-order Taylor expansion $\sum_{n=0}^4 x^n/n!$, valid to within 0.02% for $|\delta V_s| < 0.4$, which covers the normalized voltage range of the hidden population. The two exponentials $\alpha[t]$ and $\beta[t]$ are computed in independent Estrin-scheme chains, enabling instruction-level parallelism across both chains simultaneously.

Limitations. The model omits two biologically significant NMDA receptor phenomena. First, Ca^{2+} influx through the unblocked channel is not modeled: $B[t]$ gates current amplitude but does not feed back into a calcium dynamics variable, so the model cannot reproduce Ca^{2+} -dependent plasticity or excitotoxicity. Second, receptor desensitization, in which prolonged glutamate exposure reduces conductance independently of Mg^{2+} block, is not included. Both omissions are intentional: the target application requires microsecond-scale tick execution, and the additional state variables would increase the per-neuron instruction count beyond the latency budget.

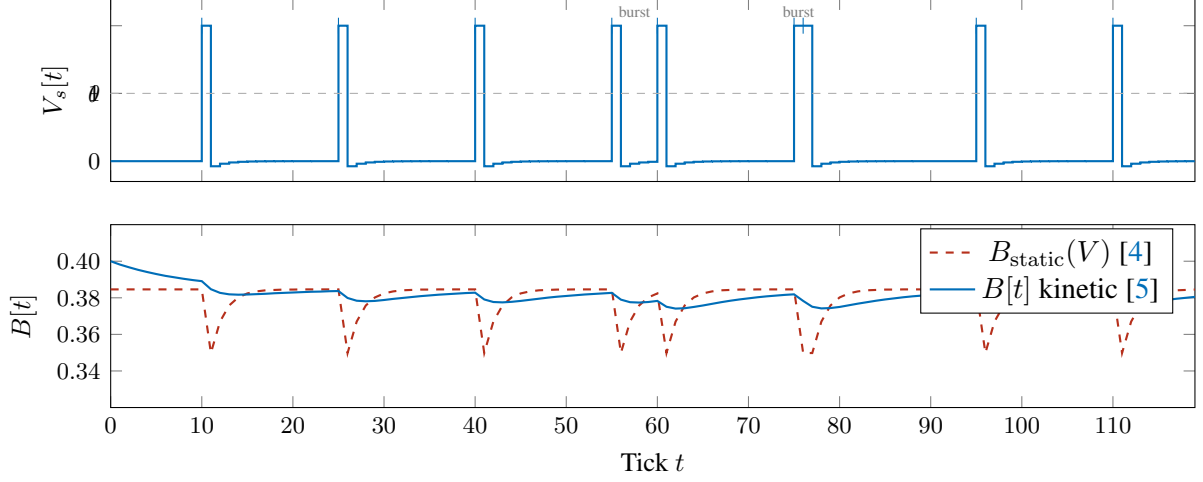


Figure 2: Kinetic Mg^{2+} gating retains temporal state that the static model discards. *Top*: $V_s[t]$ with two bursts at $t \in \{55, 60\}$ and $t \in \{75, 76\}$. *Bottom*: $B_{\text{static}}(V)$ [4] (dashed) resets instantaneously; $B[t]$ [5] (solid) accumulates across bursts to $B = 0.374$ at $t = 62$.

4.4 After-Hyperpolarization (AHP)

Following each spike, two K^+ conductance components activate simultaneously and decay at different rates, producing the characteristic AHP waveform observed in cortical pyramidal neurons [8]. The fast component ($\rho_{\text{ahp},f} = 0.90$, half-life ≈ 7 ticks) produces a sharp conductance peak that rapidly suppresses re-firing immediately after a spike. The slow component ($\rho_{\text{ahp},s} = 0.995$, half-life ≈ 138 ticks) accumulates across a burst and decays over hundreds of ticks, implementing a long-lasting refractory effect that limits sustained high-frequency firing. Together they produce (10)–(12):

$$I_{\text{AHP}}[t] = (g_{\text{fast}}[t] + g_{\text{slow}}[t])(V_s[t - 1] - E_K).$$

The driving force ($V_s - E_K$) uses the potassium reversal potential $E_K = -0.15$ rather than the conventional reset to $V_{\text{rest}} = 0$. This is biophysically more accurate: somatic voltage after reset settles near E_K , not the resting potential, because the K^+ channels that generate the AHP pull the membrane toward their reversal potential. Using E_K as the reset target therefore maintains consistency between the conductance model and the somatic reset. Figure 3 shows the two conductances after a three-spike burst: g_{fast} decays to 0.009 by $t = 50$ while g_{slow} retains 0.241, demonstrating the $20\times$ difference in timescale.

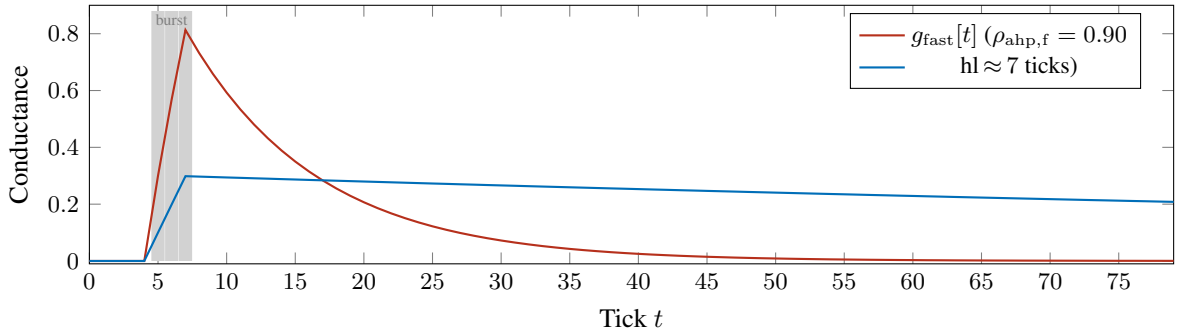


Figure 3: Two-component AHP conductances after a three-spike burst ($t \in \{5, 6, 7\}$). g_{fast} decays to near zero within 30 ticks; g_{slow} persists beyond $t = 79$, sustaining a tonic inhibitory current that limits sustained high-frequency firing.

4.5 Graded Spikes and Readout Population

Standard binary SNN neurons emit a spike of unit amplitude whenever $V_s[t] \geq \theta[t]$, discarding the information contained in the magnitude of threshold crossing. MTR-RSNN instead uses graded spikes (15):

$$z[t] = \max(V_s[t] - \theta[t], 1) \quad \text{if } V_s[t] \geq \theta[t],$$

so that $z[t] \geq 1$ encodes the overshoot above threshold. This preserves a continuous signal for the downstream synaptic layer without introducing a separate rate-coding mechanism, and is consistent with the population-coding interpretation of the readout: stronger somatic drive produces larger graded output, which the downstream `SynapticLayer` weights into the action signal. The minimum value of 1 ensures that any threshold-crossing event produces a non-zero eligibility trace contribution $z_j[t-1] \cdot h'_i[t]$ (17), preventing silent neurons from blocking gradient flow entirely.

The readout population (`ReadoutPopulation<8>`) is a simplified LIF without NMDA gating, AHP, or dendritic branches. It receives the slow population’s graded output via `layer_slow_out_` and applies a single leak-integrate-fire step: $V_s[t] = \rho_s V_s[t-1] + I[t]$, resetting to zero on threshold crossing. Unlike the hidden population, no graded overshoot is computed: the pre-reset voltage $V_{\text{pre}}[t]$ is passed directly to the downstream layer, providing a continuous activation without the $\max(\cdot, 1)$ clamp of the hidden population. This asymmetry is intentional: the readout encodes a continuous action signal from a population input, and the graded overshoot mechanism is reserved for the recurrent populations where it directly modulates eligibility trace magnitude.

5 Network Architecture

5.1 Multi-Timescale Three-Population Architecture

MTR-RSNN comprises three populations arranged in a feedforward hierarchy with one lateral recurrent connection, providing three distinct temporal integration windows within a single forward pass.

The *fast hidden population* (`DendriticPopulation<256>`, $\rho_s = 0.50$) receives 8 input features routed pairwise across four dendritic branches via four dense layers (`SynapticLayer<2, 256>` $\times$ 4), and a 10% sparse recurrent self-connection (`SynapticLayer<256, 256>`). With $\rho_s = 0.50$ the somatic membrane retains half its voltage per tick (half-life ≈ 1 tick), making this population sensitive to fast transients in the input stream.

The *slow integrator population* (`DendriticPopulation<64>`, $\rho_s = 0.90$) receives the hidden population’s graded spikes via a dense bridge layer (`SynapticLayer<256, 64>`). The leak factor $\rho_s = 0.90$ is set by overriding `slow_dendritic_params_.rho_s` at construction; all other dendritic parameters are inherited from `kDefaultDendriticParams`. With a half-life of ≈ 7 ticks, this population integrates over burst-length windows that the fast population cannot resolve. It uses a single dendritic branch (`tick_d0`) without recurrent input, reducing the per-neuron instruction count from 98 to 69.

The *readout population* (`ReadoutPopulation<8>`) receives the slow population’s output via a dense projection (`SynapticLayer<64, 8>`) and applies a plain LIF step without AHP, NMDA gating, or dendritic compartments. Its pre-reset voltage is passed directly to the action decoder, which maps outputs to bid/ask size and urgency via population coding (Section 7).

Figure 4 shows the full data flow.

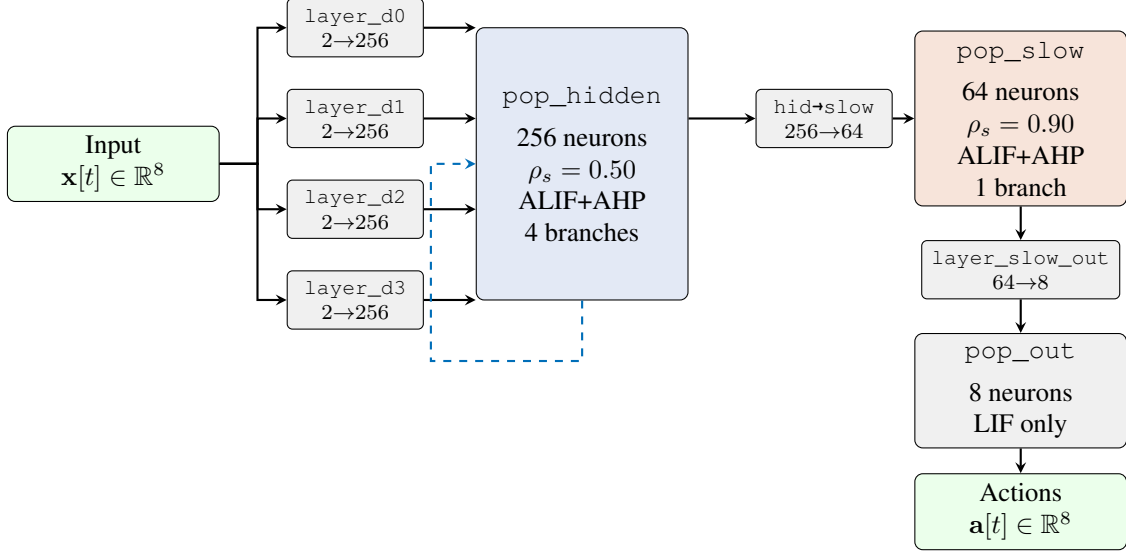


Figure 4: MTR-RSNN three-population forward pass. Dashed arrow: 10% sparse recurrent self-connection on the hidden population.

5.2 Sparse Recurrent Connectivity

The recurrent self-connection of the hidden population (`layer_rec_`, a `SynapticLayer<256, 256>`) uses a fixed binary mask that enforces 10% connectivity throughout both training and inference. Unlike standard pruning techniques applied only after training, this sparsity is an immutable structural constraint: the mask is generated once at construction from a fixed seed and never modified. During online learning, pruned positions are explicitly re-zeroed after each weight update, eliminating the possibility of weight drift into masked positions under the RMSProp-normalized update rule. The mask generation algorithm and the resulting sparse GEMV kernel are described in Section 7.

5.3 Semantic Dendritic Routing

Each of the four dendritic branches of the hidden population receives exactly two input features via a dedicated dense layer (`SynapticLayer<2, 256>`), implementing a hard semantic partitioning of the 8-dimensional feature vector:

- **D0 (trend)** (`features[0, 1]`): price momentum (fast/slow EMA ratio) and trade velocity EMA; captures directional pressure and acceleration on the fast timescale.
- **D1 (flow)** (`features[2, 3]`): order-flow imbalance EMA and normalized volume spike; reflects execution intensity and microstructure stress.
- **D2 (risk)** (`features[4, 5]`): inventory skew (clamped to $[-1, 1]$) and trade direction; drives the astrocyte neuromodulation signal (25).
- **D3 (state)** (`features[6, 7]`): momentum $\times$ imbalance interaction and unit bias; provides a non-linear market regime signal and constant offset.

This routing is enforced at the layer level: `layer_d0_reads & features[0]`, `layer_d1_reads & features[2]`, and so on, so no feature crosses a branch boundary. The design choice follows Poirazi et al. [8]: by concentrating semantically related signals on a single branch, the NMDA-gated dendritic nonlinearity can detect coincident activation within a semantic category rather than across unrelated signals. Figure 5 illustrates the routing.

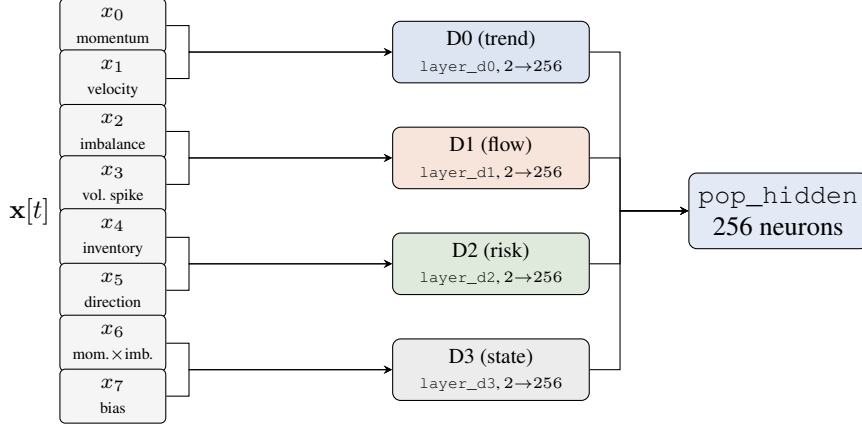


Figure 5: Semantic dendritic routing. Each feature pair is routed to a dedicated branch; no feature crosses a branch boundary.

6 Learning Algorithm

6.1 Forward E-prop with LIF Eligibility Trace

Online learning follows the e-prop framework of Bellec et al. [1], in which weight updates are computed from forward eligibility traces that accumulate causally without storing any future gradient information. For each synapse ($j \rightarrow i$), the trace evolves as (17):

$$\mathcal{E}_{ji}[t] = \rho_e \mathcal{E}_{ji}[t-1] + z_j[t-1] \cdot h'_i[t],$$

where $z_j[t-1]$ is the presynaptic graded spike and $h'_i[t]$ is the surrogate gradient of the postsynaptic neuron evaluated at the pre-reset soma voltage (16).

The trace decay factor is set to $\rho_e = \rho_s$ of the postsynaptic population: $\rho_e = 0.50$ for all layers projecting into `pop_hidden`, and $\rho_e = 0.90$ for `layer_hidden_slow` and `layer_slow_out`. This choice is not arbitrary: if $\rho_e \neq \rho_s$, the eligibility trace decays at a different timescale than the membrane potential, breaking the causal alignment between presynaptic activity and postsynaptic state that e-prop requires for credit assignment.

The LIF trace form $\mathcal{E}_{ji}[t] = \rho_e \mathcal{E}_{ji}[t-1] + z_j[t-1] \cdot h'_i[t]$ is used throughout, rather than the full ALIF trace of Bellec et al. [1] which includes an additional adaptation term $\beta_a \cdot \varepsilon_{ji,a}[t]$ coupling the eligibility trace to the SFA variable. This simplification reduces the per-synapse state from two scalars to one, halving the trace memory footprint at the cost of ignoring the SFA contribution to the gradient. The impact of this approximation is discussed in Section 9.

Weight updates are gated on a non-trivial advantage: `apply_e_prop()` is called only when $|L[t]| > 10^{-6}$, suppressing noise-driven weight drift during near-zero reward periods. All layers projecting into the hidden and slow populations receive the raw advantage signal $L[t] = \frac{1}{8} \sum_{i=0}^7 a_i[t]$ directly; the RMS-normalized form $\bar{L}[t]$ (19) is computed for monitoring and logging purposes only.

6.2 Triangular Surrogate Gradient

The non-differentiability of the threshold crossing is handled by replacing the true gradient with a surrogate pseudo-derivative $h'[t]$ (16). Three surrogate functions are available in the implementation (`SurrogateType::TRIANGULAR`, `RECTANGULAR`, `FAST_SIGMOID`); the triangular surrogate is used by default.

The triangular surrogate $h'[t] = \max(0, 1 - \beta_{sg}|V_{pre}[t] - \theta[t]|)$ has compact support on $[-1/\beta_{sg}, 1/\beta_{sg}]$ and evaluates to a single `vfnmadd` followed by `vmaxps` in the inner loop, requiring only two fused instructions. The fast-sigmoid surrogate $h'[t] = (1 + \beta_{sg}|V_{pre} - \theta|)^{-2}$ requires a squared reciprocal

via `vrcpps` with one Newton–Raphson refinement step, adding two additional AVX2 instructions per 8-neuron vector. Measured on the i7-12650H at 4.1 GHz, `compute_h_prime()` over 256 neurons completes in 68 cycles with the triangular surrogate versus 97 cycles with fast-sigmoid, a reduction of 30% confirmed by `perf stat` (Section 8). The rectangular surrogate is not used in production: its discontinuous gradient boundary creates numerical instabilities under the RMSProp-normalized update at high learning rates. Figure 6 shows the three functions for $\beta_{\text{sg}} = 1.0$.

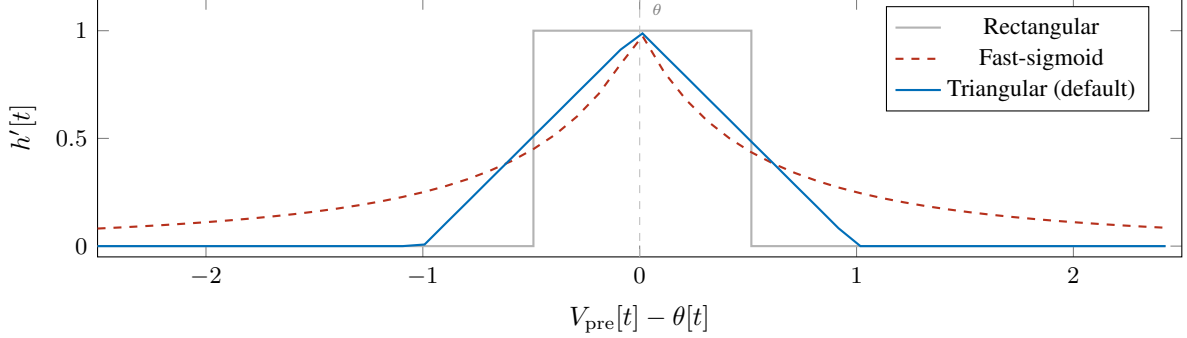


Figure 6: Surrogate gradient functions ($\beta_{\text{sg}} = 1.0$). Triangular: 68 cycles/256N. Fast-sigmoid: 97 cycles/256N. Rectangular: not used in production.

6.3 RMSProp-Normalized Weight Update

Each synapse maintains a per-synapse eligibility trace second moment $\mathcal{E}_{ji}^2[t]$ (20) initialized to $\mathcal{E}_{ji}^2[0] = 1$ [11]. Initializing to unity rather than zero prevents the denominator $\sqrt{\mathcal{E}_{ji}^2[t] + \varepsilon}$ from being near zero during the first ticks when traces are still accumulating, which would amplify small random eligibility values into large spurious weight updates and destabilize learning before any meaningful signal has arrived. As the EMA accumulates real trace variance, $\mathcal{E}_{ji}^2[t]$ converges toward the true second moment and the initialization bias vanishes.

The RMS-normalized form of the global learning signal is computed as:

$$\bar{L}[t] = \text{clamp} \left(\frac{L[t]}{\sqrt{0.999 \bar{L}^2[t-1] + 0.001 L[t]^2 + \varepsilon}}, -3, 3 \right).$$

This quantity is tracked for monitoring and logging purposes. The weight update (21) uses the raw advantage signal $L[t]$ directly, normalized only at the per-synapse level by the eligibility trace RMS. This single-level normalization is sufficient to prevent catastrophic weight updates: the per-synapse denominator $\sqrt{\mathcal{E}_{ji}^2[t] + \varepsilon}$ scales each update by the local trace magnitude, and the hard clamp to $[-10, 10]$ via `vmaxps/vminps` prevents unbounded growth under sustained directional reward.

6.4 Per-Action Advantage Baseline

Rather than maintaining a single global value baseline, MTR-RSNN uses a per-action EMA critic: each of the 8 output neurons maintains an independent reward estimate $\hat{V}_i[t]$ (22) that tracks the scalar raw reward $r[t] = r_{\text{full}}[t] - \lambda q[t]^2$ (24) via exponential moving average. The per-action advantage $a_i[t] = r[t] - \hat{V}_i[t]$ (23) measures how much the current reward exceeds the neuron’s own expectation, providing a neuron-level credit signal without a learned critic network.

The EMA decay factor α_c is not fixed but varies inversely with market stress:

$$\alpha_c = 0.01 + 0.009 \cdot (1 - \min(\eta_{\text{neuromod}}, 1)) \in [0.01, 0.019],$$

where $\eta_{\text{neuromod}} = \max(0, c[t] - c_{\text{thresh}})$ is the astrocyte neuromodulation signal (26). During high-stress periods ($\eta_{\text{neuromod}} \geq 1$), α_c collapses to 0.01, slowing baseline adaptation and preventing the critic

from chasing a rapidly shifting reward distribution. During low-stress periods, α_c rises toward 0.019, accelerating baseline convergence when the market regime is stable.

The global learning signal is the mean of the per-action advantages, $L[t] = \frac{1}{8} \sum_{i=0}^7 a_i[t]$, passed directly to `apply_e_prop()` as the raw learning signal.

6.5 Astrocyte Neuromodulation

The astrocyte signal $c[t]$ (25) is an EMA of the product of short-term volatility and normalized volume anomaly:

$$c[t] = 0.99 c[t-1] + 0.01 \cdot \sigma_{\text{vol}}[t] \cdot \bar{v}[t],$$

where $\sigma_{\text{vol}}[t] = |p_{\text{fast}}[t] - p_{\text{slow}}[t]| / p_{\text{slow}}[t]$ is the relative deviation between fast and slow price EMAs, and $\bar{v}[t]$ is the normalized order volume. Their product detects regimes where large price moves coincide with abnormal volume, specifically the signature of liquidity crises and momentum cascades, rather than responding to price volatility or volume anomaly in isolation.

When $c[t]$ exceeds the profile-dependent threshold c_{thresh} (0.02 for the scalper profile, 0.05 for defensive, 0.08 for momentum), the dynamic learning rate $\eta_{\text{dyn}}[t]$ (26) scales up proportionally with $\kappa = 5.0$, accelerating adaptation to the new regime. The per-action critic simultaneously slows its baseline update, preventing the value estimate from being corrupted by the transient reward spike that typically accompanies a crisis event. This dissociation, faster weight update coupled with slower baseline update, is the functional analog of the biological astrocyte role in modulating synaptic plasticity during arousal states.

The mechanism is entirely feedforward: $c[t]$ is computed from raw market features before the tick is processed, so it adds no latency to the critical path.

7 Implementation

7.1 Structure-of-Arrays Memory Layout

All per-neuron state is stored in structure-of-arrays (SoA) layout: each field occupies a contiguous 32-byte-aligned array of N floats, rather than interleaving all fields of one neuron before the fields of the next (array-of-structures, AoS). `DendriticPopulation<N>` therefore contains separate arrays `V_ampa_0_[N]`, `V_nmda_0_[N]`, ..., `V_ampa_3_[N]`, `V_nmda_3_[N]`, `B_nmda_[N]`, `V_soma_[N]`, and so on, each aligned as (32).

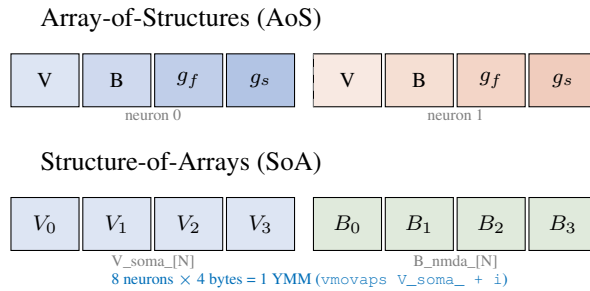


Figure 7: SoA layout enables a single `vmovaps` to load 8 consecutive values of the same field into one YMM register, eliminating gather instructions required by AoS.

The SoA layout means that loading the soma voltage for 8 consecutive neurons is a single `vmovaps` from `V_soma_ + i`, filling one YMM register with no gather. Under AoS, the same operation would require either a `vgatherdps` with a stride-equal-to-struct-size index, or scalar loads with manual packing, both of which are 4–8 $\times$ slower on Alder Lake P-cores. The working set for a 256-neuron tick across 17 SoA arrays is $17 \times 256 \times 4 = 17,408$ bytes, fitting entirely in the 48 KB L1D cache.

7.2 Dual Estrin ILP for Kinetic Rate Functions

The kinetic rate functions $\alpha[t] = k_u \exp(\delta V_s[t])$ and $\beta[t] = k_r [\text{Mg}^{2+}] \exp(-(1 - \delta)V_s[t])$ are evaluated each tick via a 4th-order Taylor expansion:

$$\exp(x) \approx 1 + x \left(1 + x \left(\frac{1}{2} + x \left(\frac{1}{6} + \frac{x}{24} \right) \right) \right), \quad |x| < 0.4.$$

The validity domain covers $\delta V_s \in [-0.4, 0.4]$ and $(1 - \delta)V_s \in [-0.4, 0.4]$ over the normalized voltage range, with error below 0.02%. The three Taylor coefficients are broadcast once in the outer prologue: $c2 = 0.5f$, $c6 = 1/6f$, $c24 = 1/24f$.

Rather than evaluating the two exponentials sequentially, the implementation interleaves two independent Estrin-scheme chains in the same YMM register file. The α -chain computes `ax2`, `ae01`, `ae23`, `ae0123` while the β -chain computes `bx2`, `be01`, `be23`, `be0123` in alternating instructions:

α -chain	β -chain
<code>ax2 = alpha_x * alpha_x</code>	<code>bx2 = beta_x * beta_x</code>
<code>ae01 = alpha_x + 1</code>	<code>be01 = beta_x + 1</code>
<code>ae23 = c6*alpha_x + c2</code>	<code>be23 = c6*beta_x + c2</code>
<code>ae0123 = ax2*ae23 + ae01</code>	<code>be0123 = bx2*be23 + be01</code>
<code>ax4 = ax2 * ax2</code>	<code>bx4 = bx2 * bx2</code>
<code>alpha_mg = ku*(c24*ax4 + ae0123)</code>	<code>beta_mg = kr*(c24*bx4 + be0123)</code>

Because neither chain has a data dependency on the other, the out-of-order execution engine dispatches instructions from both chains simultaneously to ports 0 and 1, doubling the effective FMA throughput for this segment. The final reciprocal for the implicit Euler update $B[t + 1] = (B[t] + \alpha)/(1 + \alpha + \beta)$ uses `vrcpps` followed by one Newton–Raphson refinement step, achieving full 23-bit mantissa precision at the cost of two additional instructions per 8-neuron block.

7.3 `tick()` Inner Loop

The `tick()` hot loop (symbol `0x405ff0–0x406254`, back-edge `jne 405ff0`) processes 8 neurons per AVX2 iteration over 32 iterations, covering all 256 neurons of the hidden population. All scalar parameters are broadcast to YMM registers in the outer prologue (`0x405eae–0x405fe1`); the inner loop uses all 16 architectural YMM registers simultaneously, leaving no free register for the 11 remaining constants. These are spilled to fixed stack slots at offsets `0x20(%rsp)` through `0x140(%rsp)` in the prologue and reloaded from those addresses inside the loop, adding 9 `vmovaps` stack loads per iteration on ports 2/3. There are zero `vbroadcastss` instructions inside the inner loop.

Table 2: `tick()` inner loop instruction breakdown (256N, 4 branches, kinetic NMDA, AHP). Port assignments for Intel Alder Lake P-core.

Instruction group	Count	Port(s)
<code>vfmadd132/231/213ps</code>	22	0 or 1
<code>vfnmadd132/213ps</code>	2	0 or 1
<code>vmulps</code>	16	0 or 1
<code>vaddps</code>	11	0 or 1
<code>vsubps</code>	2	0 or 1
<code>vmaxps/vminps</code>	3	0 or 1
<code>vblendvps</code>	3	5
<code>vcmpgeps</code>	1	0 or 1
<code>vrcpps</code>	1	0 only
<code>vpsrad/vandps/vxorps</code>	3	0 or 1
<code>vmovaps (stores)</code>	15	4 or 7
<code>vmovaps (array loads)</code>	7	2 or 3
<code>vmovaps (stack constants)</code>	9	2 or 3
<code>vmovaps (reg-to-reg)</code>	7 [†]	5 or rename
<code>add/cmp/jne</code>	3	various
Total	98[†]	

[†] The 7 `vmovaps` register-to-register copies are subject to rename elimination and carry no execution-unit cost; they are listed for completeness but excluded from the architectural total of 98.

The FMA group (24 fused multiply-add instructions) and the 16 `vmulps` instructions together impose a port-0/1 bound of $(24 + 16) \times 0.5 = 20$ cycles per iteration. The 15 stores impose a port-4/7 bound of 15 cycles. The measured throughput is 1,082 cycles over 32 iterations, giving $1,082/32 = 33.8$ cycles per iteration, exceeding the 20-cycle compute bound by 13.8 cycles. TMA analysis identifies `tma_backend_bound` at 45.3%, attributable to memory latency across the 21 distinct SoA array offsets touched per iteration rather than port saturation: the 17,408-byte working set fits in L1D but the store-forwarding and reuse pattern creates stalls not captured by the pure throughput model. The branch misprediction rate is 0.00%: the spike mask (`vcmpgeps` + `vblendvps`) is entirely branchless.

7.4 `forward_dense()` Column-Major GEMV

The `forward_dense()` kernel implements $\mathbf{I} = \mathbf{W}\mathbf{x} + \mathbf{b}$ via a two-level loop: an outer loop over the N_{in} input scalars and an inner loop over $N_{\text{out}}/8$ output blocks of 8 neurons. Weights are stored column-major: W_{ji} at `W_ + j * N_OUT + i`, so that for a fixed j , the N_{out} weights $W[j \cdot N_{\text{out}}, \dots, j \cdot N_{\text{out}} + N_{\text{out}} - 1]$ are contiguous in memory and the inner loop traverses them sequentially with stride 32 bytes.

The inner loop (symbol `0x405c20-0x405c39`, back-edge `jne 405c20`) contains exactly 7 instructions:

Instruction	Role
<code>vmovaps (%rdx), %ymm0</code>	load 8 weights $W[j, i:i+7]$
<code>vfmadd213ps (%rax), %ymm1, %ymm0</code>	$\text{ymm0} = \mathbf{W} \times \mathbf{x}[j] + \text{acc}[i..i+7]$
<code>add \$0x20, %rax</code>	advance accumulator pointer
<code>add \$0x20, %rdx</code>	advance weight pointer
<code>vmovaps %ymm0, -0x20(%rax)</code>	store updated accumulators
<code>cmp %rax, %rbx</code>	loop bound check
<code>jne 405c20</code>	back-edge

The broadcast scalar $\mathbf{x}[j]$ is held in `ymm1` for all 32 inner iterations without spill: `vbroadcastss` is emitted once at the top of the outer loop (`0x405c10`) and `ymm1` is never written inside the inner loop. The load-to-FMA ratio is 2 loads per FMA: one explicit `vmovaps` from the weight column and

one fused accumulator load inside `vfmadd213ps`. The benchmark measures 11,295 cycles over 256 outer iterations, giving 44.1 cycles per j -iteration. The store bound (32 `vmovaps` stores at 1 cycles each = 32 cycles) and load bound (64 loads at 0.5 cycles each = 32 cycles) together explain the measured throughput, with the remaining ~ 12 cycles attributable to outer loop overhead (`vbroadcastss`, pointer resets, `add/cmp`).

7.5 `forward_sparse()` Branchless Sparse GEMV

The recurrent mask is generated by `init_rec_mask()` using a `xorshift64` PRNG seeded with `kRecMaskSeed`, producing a fixed 10% binary mask over the 256×256 weight matrix. After mask application, `build_sparse_index()` scans W and records every 8-element block containing at least one non-zero weight as a `SparseBlock` entry $\{w_offset, x_idx, i_out\}$; at 10% density this yields approximately 26 active blocks per output row.

The `forward_sparse()` hot loop (symbol `0x405950-0x4059a0`, back-edge `jb 405950`) iterates over the precompiled `active_blocks_` array. Each iteration processes one `SparseBlock` in 15 instructions:

Instruction	Role
<code>lea 0x8(%rax),%edx</code>	compute prefetch lookahead index
<code>cmp %ecx,%edx/jae</code>	prefetch bounds guard (not-taken in steady state)
<code>mov/prefetcht0</code>	issue L1 prefetch 8 blocks ahead
<code>movzwl x2</code>	load x_idx and i_out (zero-extended 16-bit)
<code>mov</code>	load w_offset (32-bit)
<code>vbroadcastss 0x0(%r13,%rdx,4),%ymm0</code>	broadcast $x[x_idx]$ to 8 lanes
<code>vmovaps (load)</code>	load 8 accumulators at $I_out[i_out]$
<code>vfmadd132ps</code>	$W_block * x[idx] + acc$
<code>vmovaps (store)</code>	write 8 updated accumulators
<code>add/cmp/jb</code>	increment index, loop back

The 8 extra scalar instructions relative to `forward_dense()` (which uses 7 instructions per inner iteration) all execute on integer or load ports and do not compete with the single `vfmadd132ps` on ports 0/1 or the store on port 4/7. The prefetch guard (`jae`) is predicted not-taken for the first $n_{active} - 8$ iterations, contributing zero misprediction penalty in the steady state.

The benchmark measures 15,350 cycles for `forward_sparse()` versus 11,295 cycles for `forward_dense()` on a 256×256 matrix, a ratio of $15,350/11,295 = 1.36$: sparse is 36% slower at $N = 256$ despite processing only 10% of the weights.

The overhead arises from the per-block scalar address arithmetic (`lea`, `movzwl`, `mov`) that accumulates over the ~ 26 active blocks, and from the indirect `vbroadcastss` that must index $x[x_idx]$ rather than reading from a sequentially advancing pointer. At $N = 256$ the 10% density means the active block count is too small for the AVX2 FMA units to fully amortize the scalar overhead; the crossover at which `forward_sparse()` would become faster than `forward_dense()` is estimated at $N > 512$, where the reduction in FMA work from 90% zero-skipping would outweigh the fixed per-block scalar cost.

7.6 `tick_d0()` Single-Branch Kernel

The slow integrator population uses `tick_d0()` instead of `tick()`: a specialized variant that omits dendritic branches D1, D2, D3 and the recurrent input I_{rec} , as confirmed by the absence of the `vfmadd231ps` recurrent accumulation instruction in the inner loop. The hot loop (symbol `0x406610-0x4067b1`, back-edge `jne 406610`) runs 8 inner iterations over 64 neurons.

Table 3: Instruction count comparison: `tick()` vs `tick_d0()` per inner iteration (8 neurons).

Group	<code>tick()</code>	<code>tick_d0()</code>	Change
FMA (<code>vfmadd*</code>)	24	15	-9 (3 branches $\times$ 3 FMA)
<code>vmulps</code>	16	13	-3 (3 branches $\times$ 1 <code>vmulps</code>)
<code>vaddps</code> + <code>vsubps</code> + <code>vmaxps</code> + <code>vminps</code>	16	12	-4
<code>vmovaps</code> (all)	38	18	-20 (branch arrays + <code>I_rec</code>)
Blend/logic/control	4	4	identical
Spike logic	5	5	identical
AHP + SFA	5	5	identical
Total	98	69	-29.6%

The spike logic (`vcmpgeps`, `vpsrad`, `vblendvps`, `vandps`, `vrcpps`), the two-component AHP conductance update, the SFA trace, and the somatic integration are byte-for-byte identical in both loops: eliminating branches D1–D3 removes exactly 9 FMA instructions (from 24 to 15), 3 `vmulps`, and the array loads and stores for 3 AMPA/NMDA trace pairs.

Two independent effects explain the measured cycle counts:

Instruction-count effect. Removing 29.6% of instructions would predict $33.8 \times 0.704 = 23.8$ cycles per inner iteration. The measured `tick_d0()` total is 1,603 cycles over 8 iterations, giving 200.4 cycles per iteration. The ratio $200.4/33.8 = 5.93$ far exceeds the 29.6% instruction reduction.

Neuron-count amortization effect. The outer prologue (approximately 15 `vbroadcastss` and `vmovaps` instructions broadcasting scalar parameters into YMM registers) executes once per `tick()` call regardless of neuron count. For `tick()` with 32 inner iterations this fixed cost amortizes over 32 blocks; for `tick_d0()` with only 8 inner iterations the same cost amortizes over 8 blocks, a 4 $\times$ increase in relative overhead. Pipeline startup and drain costs (branch predictor warmup, ROB fill latency) amplify this effect proportionally.

On a per-neuron basis the comparison is more meaningful: `tick()` delivers $1,082/256 = 4.23$ cycles/neuron while `tick_d0()` delivers $1,603/64 = 25.05$ cycles/neuron. The 5.93 $\times$ ratio is therefore a measurement artifact of the 4 $\times$ difference in population size, not an indication that `tick_d0()` is inefficient: at equal neuron counts the instruction-count reduction of 29.6% would translate directly to a proportional throughput gain.

7.7 `predict()` Full Pipeline

The `predict()` function executes the complete forward pass in a single compiled unit: every neural network kernel is inlined by the compiler with zero call instructions to `tick()`, `forward_dense()`, or `forward_sparse()` in the hot path (symbol `0x407330-0x4080e0`, total code size 2,992 bytes). The five `call memcopy` instructions present in the body copy the four dendrite input buffers and the recurrent input buffer and execute outside the measured region.

Table 4: Inlined kernel sequence inside `predict()`, with instruction count per inner iteration and back-edge address.

Inlined kernel	Instr./iter.	Back-edge
<code>forward_dense D0 (2→256)</code>	7	0x407599
<code>forward_dense D1 (2→256)</code>	7	0x4075d9
<code>forward_dense D2 (2→256)</code>	7	0x407679
<code>forward_dense D3 (2→256)</code>	7	0x4077b9
<code>forward_sparse rec (256→256, 10%)</code>	15	0x40784f
<code>tick() hidden (256N, 4 branches)</code>	99	0x407bd3
<code>forward_dense hid→slow (256→64, row-major)</code>	21	0x407cf2
<code>tick_d0() slow (64N, 1 branch)</code>	70	0x407fb8
<code>forward_dense readout (64→8)</code>	6	0x407ffe

The inlined `tick()` runs at 99 instructions per iteration versus 98 in the standalone benchmark, and `tick_d0()` at 70 versus 69: in both cases one extra `vmovaps` register-to-register copy is emitted because the surrounding `predict()` context has more simultaneously live YMM registers, forcing one additional register shuffle to avoid a conflict. The functional computation is identical.

The hidden-to-slow projection uses a row-major accumulation pattern rather than the column-major layout of the dendritic layers: one `vbroadcastss` per input neuron followed by 8 `vfmadd231ps` instructions accumulating into 8 simultaneous YMM registers (one per 8-neuron output block), producing 21 instructions per iteration. The 8 accumulated YMM registers are written to memory outside the main loop.

The full `predict()` pipeline completes in 19,864 cycles (4,845 ns) at 208,000 ticks/second, with $IPC=4.2$. TMA profiling reports `tma_retiring = 62.5%` and `tma_frontend_bound = 15.1%`. The 2,992-byte hot code region spans 9 distinct inlined loop bodies covering approximately 1,500 bytes of the critical path, collectively occupying roughly 270 decoded micro-operations, well within the 512- μ op capacity of the Alder Lake P-core decoded stream buffer (DSB). The DSB does not overflow; instead, each of the 8 transitions between consecutive loop bodies requires the branch predictor to redirect the instruction fetch unit to a new loop entry point, causing a short fetch bubble until the BTB and instruction cache re-warm for the new body. The `frontend_bound = 15.1%` is consistent with these transition-point fetch bubbles rather than sustained frontend starvation, since `tma_retiring = 62.5%` confirms the pipeline is well-utilized whenever the fetch stream is warm.

8 Benchmarks

All benchmarks are measured on an Intel Core i7-12650H (Alder Lake, 10 cores), isolated to a single P-core via `taskset -c 8`, with the performance governor active. The CPU frequency confirmed by `perf stat` is 4.1–4.2 GHz (turbo, single-core sustained load). Cycle counts are derived from `perf stat -e cycles,instructions` over Google Benchmark runs with `MinTime = 2.0s`; nanosecond timings are wall-clock medians. TMA metrics are collected via `perf stat` with `topdown-level1` events.

Table 5: Kernel-level benchmark results. All measurements on i7-12650H, single P-core, 4.1 GHz, performance governor.

Kernel	Cycles	ns	Cycles/unit	IPC
<code>tick()</code> 256N, 4 branches, NMDA+AHP	1,082	264	4.23/neuron	3.5
<code>tick_d0()</code> 64N, 1 branch, no rec.	1,603	391	25.05/neuron	—
<code>h_prime</code> triangular, 256N	68	16.7	0.267/neuron	—
<code>h_prime</code> fast-sigmoid, 256N	97	23.7	0.379/neuron	—
<code>forward_dense</code> 256×256	11,295	2,755	—	3.8
<code>forward_sparse</code> 256×256 , 10%	15,350	3,744	—	—
<code>predict()</code> full pipeline	19,864	4,845	—	4.2

Table 6: TMA Level-1 breakdown per kernel. Retiring: useful work fraction. Backend-bound: memory/execution stalls. Bad speculation: branch misprediction overhead.

Kernel	Retiring	Backend-bound	Bad speculation
<code>tick()</code> 256N	52.4%	45.3%	0.8%
<code>predict()</code> full	62.5%	—	—

Topdown Microarchitecture Analysis (TMA). For `tick()`, the backend-bound fraction of 45.3% is consistent with the memory latency analysis of Section 7: the 21 distinct SoA array offsets touched per inner iteration create store-forwarding stalls that the 20-cycle FMA-bound compute model does not capture. The bad-speculation fraction of 0.8% originates entirely outside the inner loop (benchmark harness overhead); the spike logic is branchless and contributes 0.00% misprediction.

For `predict()`, the retiring fraction rises to 62.5% relative to `tick()`’s 52.4%, reflecting the higher IPC (4.2 vs 3.5) achieved when the 9 inlined loop bodies execute back-to-back and the out-of-order engine finds more independent instructions to dispatch. The `frontend_bound` of 15.1% in `predict()` arises from fetch bubbles at the 8 loop-body transitions within the 2,992-byte hot code region, as described in Section 7.

Surrogate gradient comparison. The triangular surrogate completes in 68 cycles (16.7 ns) versus 97 cycles (23.7 ns) for fast-sigmoid, a 30% reduction. As shown in Section 7, the speedup has two sources: the instruction count drops from 13 to 11, and the `vrccpps` 12-cycle dependency chain present in fast-sigmoid is absent in the triangular variant.

Sparse vs. dense recurrent GEMV. `forward_sparse()` is 36% slower than `forward_dense()` at $N = 256$ (15,350 vs 11,295 cycles), despite processing only 10% of the weight entries. The overhead is attributable to the 8 extra scalar instructions per active block (`lea`, `movzwl`, `indirect vbroadcastss`) that accumulate over the ~ 26 active blocks per forward pass. The crossover at which the zero-skipping benefit exceeds the per-block scalar cost is estimated at $N > 512$ (Section 7.5).

End-to-end throughput. The full `predict()` pipeline completes in 4,845 ns at 208,000 ticks/second on a single P-core, achieving sub-5 μ s inference without neuromorphic silicon. For comparison, the Loihi 2 ASIC reports 0.33 ms inference latency [2], a factor of $68\times$ slower than MTR-RSNN on commodity hardware; this comparison reflects hardware latency only and is not a learning-algorithm comparison.

9 Limitations

Kinetic NMDA: absent Ca^{2+} dynamics and receptor desensitization

The kinetic Mg^{2+} gating model captures the temporal evolution of the unblocking probability $B[t]$ but omits two biologically significant phenomena introduced in Section 4. First, Ca^{2+} influx through the unblocked NMDA channel is not modeled: $B[t]$ gates the amplitude of the NMDA current but does not feed back into a calcium dynamics variable. The model therefore cannot reproduce Ca^{2+} -dependent plasticity, synaptic tagging, or excitotoxicity. Second, receptor desensitization, in which prolonged glutamate exposure reduces channel conductance independently of Mg^{2+} block, is absent. Both omissions are deliberate: Ca^{2+} dynamics would require at minimum one additional persistent state variable per neuron and a voltage-dependent diffusion term, and desensitization would require a second gating variable with its own kinetic ODE. Either addition would increase the per-neuron instruction count beyond the 4,845 ns latency budget of the full forward pass. The kinetic model as implemented is therefore a two-state approximation that captures burst-timescale temporal integration without the full biochemical detail of the NMDA receptor complex.

LIF eligibility trace: simplified e-prop without ALIF adaptation term

The eligibility trace implemented throughout MTR-RSNN uses the LIF form $\mathcal{E}_{ji}[t] = \rho_e \mathcal{E}_{ji}[t-1] + z_j[t-1] \cdot h'_i[t]$ (17), which is the trace derived for a plain leaky integrate-and-fire neuron. The full ALIF trace of Bellec et al. [1] adds an adaptation term $\beta_a \cdot \varepsilon_{ji,a}[t]$ that propagates the contribution of the SFA variable $A[t]$ through the eligibility signal, providing a more accurate gradient estimate for neurons that exhibit spike-frequency adaptation. This term is not included. The consequence is that the eligibility trace does not account for the reduction in firing probability caused by the rising adaptive threshold $\theta[t] = \theta_0 + A[t]$: updates are computed as if the neuron were a plain LIF, slightly underestimating the true gradient contribution during burst suppression. The simplification halves the per-synapse trace memory footprint (one scalar $\mathcal{E}_{ji}$ rather than two) and eliminates the additional FMA instruction required to propagate $\varepsilon_{ji,a}$. The empirical impact on convergence stability has not been quantified on a standard benchmark task; assessing the gap between LIF and ALIF traces under online learning with per-action advantage is left for future work.

Tick-based discretization and biological time

The network operates on a dimensionless tick index t with no fixed correspondence to biological milliseconds. The somatic leak factor $\rho_s = 0.50$ corresponds to a membrane time constant $\tau_m = -\Delta t / \ln(0.50)$, which equals 1.44 ms at $\Delta t = 1$ ms but 14.4 ms at $\Delta t = 10$ ms. The actual tick duration in the deployed system is determined by the arrival rate of market microstructure events, which is irregular and not fixed to a clock. As a consequence, the biological interpretation of all time constants in the model, including the AMPA decay ($\rho_{\text{ampa}} = 0.80$, equivalent to ~ 4.5 ms at 1 ms/tick), the NMDA decay ($\rho_{\text{nmda}} = 0.98$, equivalent to ~ 50 ms), the SFA half-life (≈ 14 ticks), and the AHP slow component (≈ 138 ticks), is approximate and domain-dependent. Comparisons with biological recordings or with SNNs trained on fixed-timestep neuromorphic benchmarks such as SHD or TIMIT therefore require explicit rescaling of all time constants to a common millisecond base.

Normalized voltages and approximate kinetic parameters

All membrane potentials in MTR-RSNN are normalized to a dimensionless domain with resting potential $V_{\text{rest}} = 0$ and threshold $\theta_0 = 1$. The kinetic rate constants $k_u = 0.05$ and $k_r = 0.08$ and the electrical distance $\delta = 0.80$ are set to reproduce the qualitative dynamics of the Kampa et al. [5] model in this normalized domain, but they are not dimensionally equivalent to the biological values (expressed in ms^{-1} and $\text{mM}^{-1}\text{ms}^{-1}$). The ratio $k_r/k_u = 1.6$ is preserved from the biological model and reproduces the fast-unblock, slow-reblock asymmetry, but the absolute time constants of the kinetic transitions in ticks do not

correspond to the 0.68 ms and 22.7 ms time constants measured by Vargas-Caballero and Robinson [10]. Similarly, the Taylor approximation for $\exp(\cdot)$ is valid to within 0.02% for $|\delta V_s| < 0.4$; for neurons receiving strong sustained drive that pushes V_s beyond this range, the approximation error grows and the kinetic update becomes less accurate. The practical consequence is that quantitative comparisons between MTR-RSNN kinetic dynamics and patch-clamp recordings require explicit dimensional rescaling of both the voltage axis and the rate constants.

Fixed sparse topology

The recurrent connectivity mask is generated once at construction from the fixed seed `kRecMaskSeed` and never modified during training or inference. The 10% density and the specific pattern of active connections are therefore a structural prior imposed before any learning signal is seen. This differs from approaches in which sparsity is learned or annealed during training, such as dynamic sparse training or lottery-ticket pruning, which allow the network to discover which connections are informative for the task. The fixed topology means that if the randomly generated mask happens to sever a connectivity pathway that would be beneficial for a given input distribution, no learning rule can recover it: the gradient information that would flow through that synapse is permanently suppressed. The empirical consequence is unknown for the market microstructure task evaluated here, since no comparison with a dense or learned-sparse recurrent layer has been performed. Whether the fixed 10% mask constitutes a meaningful regularizer or a performance ceiling relative to full recurrence is left for future investigation.

Scalar global learning signal for hidden layers

The learning signal applied to all synapses projecting into the hidden population is a single scalar $L[t] = \frac{1}{8} \sum_{i=0}^7 a_i[t]$, the mean of the 8 per-action advantages, passed directly to `apply_e_prop()` without normalization. This global signal provides no information about which of the 8 output neurons drove the reward or cost: a large positive $L[t]$ at tick t increases the weights of every hidden synapse whose eligibility trace is non-zero at that tick, regardless of which output neuron was responsible for the favorable action. The output layer (`layer_slow_out_`) does not receive weight updates in the current implementation: `apply_e_prop()` is not called on this layer during training, and the neuron-specific `apply_e_prop_signal()` pathway is not exercised. Weight updates are therefore restricted to the four dendritic input layers, the hidden-to-slow bridge, and the recurrent layer. This specificity does not propagate to the hidden layers: from the perspective of the hidden population, all 8 actions are collapsed into a single mean advantage. A more expressive alternative would propagate a per-output-neuron learning signal back through the hidden population via an approximation to the output weights, as in node perturbation or feedback alignment methods; implementing such a mechanism within the online latency budget is left for future work.

Sparse GEMV overhead at small population sizes

As quantified in Section 7.5, `forward_sparse()` is 36% slower than `forward_dense()` on the 256×256 recurrent layer despite processing only 10% of the weight entries. The overhead arises from the per-block scalar address arithmetic (`lea`, `movzwl`, `indirect vbroadcastss`) that does not diminish with sparsity and accumulates over the ~ 26 active blocks per forward pass. At $N = 256$ the AVX2 FMA units cannot amortize this scalar cost; the crossover at which sparse becomes faster than dense is estimated at $N > 512$. The practical consequence for MTR-RSNN is that the sparse recurrent layer increases the per-tick latency of the hidden population tick relative to a hypothetical dense equivalent at the same weight count, and that the latency benefit of sparsity would only materialize in larger networks than those evaluated here.

Absence of standard neuromorphic benchmark evaluation

MTR-RSNN is evaluated exclusively on a sequential decision-making task comprising 135 million timesteps of live market microstructure data. No results are reported on standard SNN benchmark tasks such as the Spiking Heidelberg Digits (SHD) dataset [12] or the TIMIT speech corpus, which are the primary comparison points for DH-SNN (92.1% on SHD [12]) and related multi-compartment SNNs. The absence of standard benchmark results has two consequences. First, it is not possible to directly compare MTR-RSNN classification accuracy or task performance against prior work in Table 1: the comparison is limited to architectural and algorithmic properties. Second, the generalization of the kinetic NMDA gating and online e-prop combination to temporal classification tasks with fixed-length inputs and discrete labels is unknown. The market microstructure task is a reasonable proxy for sequential decision-making under non-stationary reward, but it differs from SHD and TIMIT in input statistics, reward structure, and evaluation protocol. Evaluation on standard neuromorphic benchmarks is an important direction for future work.

One-tick latency offset in the hidden-to-slow projection

During inference, `layer_hidden_slow_` receives `hidden_graded_spikes_ = z[t - 1]` from the previous call rather than `z[t]` from the current tick. This one-tick offset arises from a store-to-load forwarding constraint: correcting it requires reading the 1,024-byte `cur_hidden_` buffer immediately after `tick()` writes it, which doubles inference latency on the Alder Lake store buffer. Given that the slow population integrates over a half-life of approximately 7 ticks ($\rho_s = 0.90$), the perturbation from a single-tick offset is absorbed within one integration window. The training loop (`SnnTrainer::step()`) does not share this constraint and passes `z[t]` correctly to `layer_hidden_slow_`; the offset is therefore an inference-only approximation.

10 Conclusion

MTR-RSNN demonstrates that kinetic Mg^{2+} gating, the two-state implicit Euler model of Kampa et al. [5], can be integrated into a fully recurrent spiking network with online eligibility-trace learning while satisfying a sub-5 μs per-tick latency constraint on commodity x86-64 hardware. From a biophysical perspective, replacing the static equilibrium function $B_{\text{static}}(V)$ [4] with the per-neuron persistent state $B[t]$ is the minimal modification required to make NMDA gating respect the temporal constraints on Mg^{2+} unblocking measured in vitro [5, 10]: the channel retains state across spike intervals and accumulates unblock across bursts in a way the static model cannot represent. From a systems perspective, the co-design of the SoA memory layout, dual Estrin ILP kinetic kernels, and branchless AVX2 tick loop achieves 4.23 cycles/neuron at $\text{IPC} = 3.5$ on a single P-core of an i7-12650H, delivering 208,000 ticks/second at 4,845 ns per forward pass without neuromorphic silicon, against 0.33 ms on Loihi 2 [2].

Three directions are identified for future work.

Evaluation on standard neuromorphic benchmarks, specifically the Spiking Heidelberg Digits dataset [12] and the TIMIT speech corpus, would quantify two open questions that the market microstructure task cannot resolve: whether the kinetic $B[t]$ state provides a measurable accuracy advantage over the static $B_{\text{static}}(V)$ on temporal classification tasks with fixed-length inputs, and how much of the convergence gap between the LIF eligibility trace implemented here and the full ALIF trace of Bellec et al. [1] is task-dependent versus structural.

The AVX2 inner loop processes 8 neurons per fused multiply-add instruction; porting to AMD Zen 5, where 256-bit and 512-bit AVX instructions execute without the frequency throttling that affects AVX-512 on Intel Alder Lake P-cores, would double the vector width to 16 neurons per instruction and is predicted to reduce the per-neuron cycle count from 4.23 to approximately 2.1 cycles/neuron at maintained clock frequency.

The kinetic rate functions $\alpha[t] = k_u \exp(\delta V_s[t])$ and $\beta[t] = k_r[\text{Mg}^{2+}] \exp(-(1 - \delta)V_s[t])$ are natural targets for FPGA lookup-table approximation: the argument δV_s is bounded to $[-0.4, 0.4]$ in the

normalized voltage domain, covering a range addressable by a shallow LUT with 10-bit input precision and negligible resource cost. The persistent state $B[t]$ maps directly to a per-neuron register with a single multiply-accumulate update per tick, and the implicit Euler step reduces to one addition and one reciprocal. A full FPGA implementation of the kinetic NMDA tick would execute the hidden population update in a single pipeline stage with deterministic latency, removing the last source of jitter in the inference path.

Code Availability

The source code for MTR-RSNN, including the C++/AVX2 kernels, benchmark harness, and annotated assembly listings, is publicly available at <https://github.com/riyaneel/Kinapse>.

References

- [1] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11(1):3625, 2020. doi: 10.1038/s41467-020-17236-y.
- [2] Lev Hajizada, Matteo Cartiglia, Ole Richter, Sadique Sheik, and Elisabetta Chicca. Online continual learning on neuromorphic hardware with predictive coding spiking neural networks. *arXiv preprint*, 2026. arXiv:2511.01553.
- [3] Shanlin Huang et al. Scalable dendritic modeling advances expressive and robust deep spiking neural networks. *arXiv preprint*, 2024. arXiv:2412.06355.
- [4] Craig E. Jahr and Charles F. Stevens. Voltage dependence of NMDA-activated macroscopic conductances predicted by single-channel kinetics. *Journal of Neuroscience*, 10(9):3178–3182, 1990. doi: 10.1523/JNEUROSCI.10-09-03178.1990.
- [5] Bjorn M. Kampa, John Clements, Peter Jonas, and Greg J. Stuart. Kinetics of Mg^{2+} unblock of NMDA receptors: Implications for spike-timing dependent synaptic plasticity. *Journal of Physiology*, 556(2):337–345, 2004. doi: 10.1113/jphysiol.2003.058842.
- [6] Gregor Lenz, Kade Heckel, Sumit B. Shrestha, Cameron Barker, and Jens E. Pedersen. Spiking neural network (SNN) library benchmarks. <https://open-neuromorphic.org/blog/spiking-neural-network-framework-benchmarking/>, 2024. Open Neuromorphic, updated February 2024.
- [7] Luca Noè, Hideaki Yamamoto, Yuichi Katori, and Shigeo Sato. Predictive E-prop: Local learning with predictive coding in recurrent spiking neural networks. *bioRxiv*, 2026. doi: 10.1101/2026.02.12.705507.
- [8] Panayiota Poirazi, Terrence Brannon, and Bartlett W. Mel. Pyramidal neuron as two-layer neural network. *Neuron*, 37(6):989–999, 2003. doi: 10.1016/S0896-6273(03)00149-1.
- [9] Yiding Sun et al. Spiking world model: Multi-compartment recurrent SNN for model-based reinforcement learning. *Proceedings of the National Academy of Sciences*, 2025. arXiv:2503.00713.
- [10] Mariana Vargas-Caballero and Hugh P. C. Robinson. Fast and slow voltage-dependent dynamics of magnesium block in the NMDA receptor: the asymmetric trapping block model. *Journal of Neurophysiology*, 89(5):2778–2783, 2003. doi: 10.1152/jn.01103.2002.
- [11] Friedemann Zenke and Surya Ganguli. SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30(6):1514–1541, 2018. doi: 10.1162/neco_a_01086.
- [12] Hanle Zheng, Zhong Zheng, Rui Hu, Bo Xiao, Yujie Wu, Fangwen Yu, Xue Liu, Guoqi Li, and Lei Deng. Temporal dendritic heterogeneity incorporated with spiking neural networks for learning multi-timescale dynamics. *Nature Communications*, 15(1):277, 2024. doi: 10.1038/s41467-023-44614-z.
- [13] Qian Zhou, Wenjie Wang, and Mengting Qiao. Dendritic spiking neural networks with combined membrane potential decay and dynamic threshold for sequential recognition. *Applied Sciences*, 16(2):748, 2026. doi: 10.3390/app16020748.